CCA EXAMINATION MANAGEMENT SYSTEM (EXAM_MANAGEMENT)

Comprehensive Capstone Final Defense Technical Dossier & Architecture Manual

Institution: City College of Angeles | Production Cloud Droplet: 165.22.100.18 (cca-exam.duckdns.org) | Stack: Flask / MySQL / Gunicorn / Nginx

The 30-Second Elevator Pitch for Your Panel:
"The CCA Examination Management System is a specialized, web-based academic platform engineered for City College of Angeles to solve the Examination Timetabling Problem (ETP)—a proven NP-hard combinatorial optimization challenge. By replacing manual spreadsheet scheduling with a deterministic Constraint Satisfaction Problem (CSP) heuristic engine featuring a 4-tier progressive relaxation ladder, the system generates 100% conflict-free timetables in under 2 seconds. It enforces institutional academic integrity through sister-section synchronization, strict anti-self-proctoring, PWD accessibility matching, equitable faculty workload rotation, and digital test questionnaire chain-of-custody tracking across 5 dedicated stakeholder roles."

1. Executive Overview & Institutional Problem Statement

Prior to the implementation of EXAM_MANAGEMENT, examination scheduling at City College of Angeles suffered from severe operational vulnerabilities inherent in manual spreadsheet coordination across departments (IBM, ICS, IEAS):

- **Resource Collisions:** Physical room double-bookings and faculty proctor schedule clashes caused exam delays.
- **Test Question Leaks (Asynchronous Exam Slots):** Different class sections taking the exact same subject were scheduled on different days or hours, allowing early-test takers to leak exam items to peers in later sections.
- **Academic Bias & Conflicts of Interest:** Instructors frequently proctored exams for subjects they personally taught.
- **Faculty Burnout & Inequity:** Proctoring duties were unevenly allocated, with some professors exceeding 6 continuous hours while others had zero shifts.
- **Logistical Friction & Security Breaches:** Physical exam questionnaires and completed answer booklets lacked chain-of-custody tracking, creating custody confusion during proctor changeovers.
- **Student Fatigue:** Students faced back-to-back major examinations without cognitive rest periods.

2. Technical Stack & 5-Tier Decoupled Architecture

The system employs a 5-tier decoupled topology engineered for high concurrency, zero timetable deadlocks, and resilient Linux cloud operations:

Layer	Component & Stack	Architectural Responsibilities & Configuration Parameters
1. Client / Presentation	Responsive Web UI (HTML5, Vanilla JS, Bootstrap 5, CSS)	Role-specific interfaces; dynamic CSRF token attachment on mutating requests; asynchronous notification ticker (/api/notifications/unread-count) designed to bypass session idle timer refresh.
2. Edge & DNS Gateway	DuckDNS Dynamic DNS & Nginx 1.18+ Reverse Proxy	Points cca-exam.duckdns.org to 165.22.100.18. Terminates TLS/SSL; enforces client_max_body_size 50M (for batch CSV uploads) and proxy_read_timeout 300s to prevent HTTP 504 timeouts during heavy CSP batch generation.
3. WSGI Server	Gunicorn 21.2.x (Pre-forked Worker Model)	Runs 3 worker processes on loopback (127.0.0.1:9999). Supervised by Linux Systemd (cca_exam.service) configured with Restart=always for sub-second crash revival.
4. Application & Logic	Python 3.12 / Flask 3.1.x (app.py, scheduler.py, logic.py)	Executes the CSP scheduling engine, role-based access control (RBAC), Bcrypt password validation, 20-minute rolling session timeout watchdog, ReportLab PDF generators, and SMTP mail dispatchers.
5. Persistence & Data	MySQL 8.x InnoDB (database.py, Connection Pool)	Managed by MySQLConnectionPool (size: 10, pool_reset_session=True) with on-demand direct-connection fallback under burst conditions. Auto-schema migration on startup.

3. Database Architecture & Complete 16-Table Data Model

The relational database schema is normalized to 3NF and structured to enforce referential integrity, audit compliance, and historical record archiving across academic terms:

Table Name	Core Entity & Key Columns	Relational Role & Operational Purpose
users	id, email, password_hash, role, is_active, institute_id, section	Stores user profiles for 5 roles; credentials hashed using Bcrypt; gates platform access via is_active verification.
institutes	id, code, name, default_room_capacity	Academic units (e.g. IBM, ICS, IEAS); establishes baseline capacity rules for classroom allocation.
instructors	id, name, email, institute_id, subjects_taught, is_pwd	Faculty invigilator directory; stores teaching subjects (tokenized) and physical disability flags for accessible room placement.
subjects	id, code, name, subject_type, institute_id, content_group_id	Course curriculum catalog; differentiates Major vs Minor subjects; assigns content_group_id for cross-section sync.
rooms	id, name, building, floor, capacity, is_pwd_accessible	Physical exam inventory; distinguishes Main Building and PAGCOR Building; tracks wheelchair accessibility.
exams	id, subject_id, room_id, instructor_id, section, exam_date_time, duration	Core timetable entries; binds subject, room, proctor, and section into a verified, conflict-free scheduling slot.
exam_settings	id, academic_year, exam_term, exam_semester, period_start, period_end	Defines global academic milestones (e.g., 2026-2027 Midterm) and active date windows for the scheduler.
system_settings	day_start, day_end, slot_interval, max_proctor_hrs, schedule_published	Global operational thresholds, SMTP mail server parameters, and timetable visibility gate (schedule_published).
proctor_availability	id, instructor_id, date, start_time, end_time	Stores faculty-submitted availability windows used by the scheduler as O(1) hard constraints.
proctor_absence	id, instructor_id, exam_id, date, reason, reported_at	Logs emergency faculty absences on exam days with timestamps and formal justifications.
relievers	id, instructor_id, date, time_slot, status	Standby proctor inventory available to department secretaries for immediate 1-click emergency substitution.
exam_collection	id, exam_id, proctor_id, status, collected_at, returned_at	Paper custody tracking: logs handover of test questionnaires and return of student answer booklets.
audit_logs	id, user_id, user_role, ip_address, action, details, timestamp	Non-repudiation audit trail compliant with ISO/IEC 27001 and NIST SP 800-53; captures all sensitive system modifications.
schedule_reset_requests	id, requested_by_id, reason, status, reviewed_by_id, reviewed_at	Dual-authorization governance table; requires two independent administrators to approve full timetable resets.
exam_archives	id, academic_year, exam_term, archive_title, snapshot_data, archived_at	Permanent historical snapshots containing complete JSON dumps of finalized exam periods for multi-year accreditation audits.
notifications	id, user_id, message, is_read, created_at	Broadcast alerts and role-specific updates displayed dynamically in the top navigation bar.

Autonomous Database Schema Synchronization (Zero-Migration Deployment):

To prevent runtime crashes during environment migrations (e.g., from local XAMPP to production Linux Droplets), ensure_database_schema() in system/database.py executes upon application launch. It programmatically inspects MySQL table catalogs, creates missing relational tables (audit_logs, exam_archives, schedule_reset_requests), and dynamically alters tables to add missing columns without manual SQL scripting.

4. The Scheduling Engine (system/scheduler.py & CSP Heuristics)

Academic examination timetabling is mathematically categorized as the **Examination Timetabling Problem (ETP)**, a proven **NP-Hard Combinatorial Optimization Problem**. A brute-force search over all permutations of courses, rooms, faculty invigilators, and timeslots is computationally intractable ($O(N!)$). To generate a verified, 100% conflict-free timetable in under two seconds, we designed a deterministic **Constraint Satisfaction Problem (CSP) Heuristic Engine with a Tiered Relaxation Ladder**.

4.1 Hard Constraints (Zero Violations Permitted):

- **1. Zero Room Double-Booking:** Ensured via interval collision check room_cal[r_id] against candidate [start, end). No two exams can occupy the same physical room simultaneously.
- **2. Zero Proctor Double-Booking:** Ensured via inst_cal[i_id]. A proctor can never be scheduled for two concurrent examination rooms.
- **3. Zero Student Overlap + 1-Hour Rest Buffer:** The function section_free(sec, s, e) checks extended interval [s - 1 hr, e + 1 hr) against section_cal. Students are guaranteed zero concurrent exams and at least 60 minutes of cognitive rest between tests.
- **4. Sister-Section Synchronization:** Sections enrolled in common courses or sharing a content_group_id are treated as an atomic scheduling cluster. The engine locks a single synchronized timestamp for all sections across parallel rooms, eliminating test question leaks.
- **5. Strict Anti-Self-Proctoring:** Faculty members are barred from invigilating exams for subjects they teach. Pre-tokenized subject sets (name, code, ID) in inst_subs_map are checked against candidate exam tokens before proctor assignment.
- **6. Proctor Availability Windows:** O(1) dictionary lookup against proctor_availability ensures instructors are only assigned during their declared working hours.
- **7. PWD Room Accessibility Matching:** Instructors with physical disabilities (is_pwd = 1) are strictly paired with classrooms flagged is_pwd_accessible = 1 (ground floor or elevator-serviced wings).
- **8. Seating Capacity Bounds:** The engine verifies room.capacity >= section_student_count (or the institute default capacity).

4.2 Soft Constraints & The 4-Tier Progressive Relaxation Ladder:

In dense exam periods with limited proctor inventories, rigid daily thresholds cause artificial scheduling deadlocks. The engine dynamically evaluates candidate slots through a 4-tier relaxation hierarchy:

Relaxation Tier	Daily Major Cap per Section	Daily Proctoring Cap per Faculty	Operational Function
Tier 1 (Strict Baseline)	Max 2 Major Exams	Max 4.0 Hours	Ideal institutional distribution; prevents student and proctor fatigue.
Tier 2 (Minor Absorption)	Max 2 Major Exams	Max 5.0 Hours	Absorbs minor faculty shortages while protecting student balance.
Tier 3 (Dense Period)	Max 3 Major Exams	Max 6.5 Hours	Accommodates high curriculum density during peak exam cycles.
Tier 4 (Fail-Safe Ceiling)	Max 3 Major Exams	Max 8.0 Hours (Absolute Ceiling)	Final fallback ensuring 100% placement rather than leaving unassigned exams.

4.3 Fair Workload Rotation & Campus Transit Heuristics:

Candidate proctors are sorted using a multi-attribute comparator: (daily_hours_worked, cumulative_term_hours, round_robin_counter). This guarantees equitable workload balance across faculty ranks. Furthermore, proctors cannot exceed 3 consecutive hours without a rest period. To prevent campus transit delays, the engine locks proctors to their assigned building (Main Building vs. PAGCOR Building) for that day, or enforces a mandatory 15-minute inter-building travel buffer.

5. Security Architecture, Resilience & Compliance

The application layer implements multi-tiered defense-in-depth security aligned with OWASP Top 10, NIST SP 800-63B, and ISO/IEC 27001 standards:

- **Cryptographic Password Storage:** Passwords are salted and hashed using Bcrypt with Blowfish cost factor 12. Plaintext credentials are never persisted.
- **NIST SP 800-63B Password Validation:** Enforces strict length, character diversity, and dictionary checks against common breached passwords.
- **Rolling 20-Minute Inactivity Watchdog:** Monitored via @app.before_request check_session_inactivity(). If idle time exceeds 1,200 seconds, the session is invalidated, an AUTH_SESSION_TIMEOUT event is logged, and the user is redirected to login. AJAX notification endpoints are explicitly exempted so background tickers do not falsely reset the idle clock.
- **Anti-Session Fixation & Leak Protection:** Upon successful login, session.clear() purges previous session tokens. Cookies are set with HttpOnly=True (blocks XSS theft), SameSite='Lax', and Secure=True in production.
- **CSRF Protection on All Mutating Requests:** Flask-WTF CSRF tokens protect all POST/PUT/DELETE forms and AJAX calls via static/js/csrf.js.
- **Flask-Limiter Brute-Force Rate Limiting:** Throttles /login to 15 requests per minute per IP address, stopping automated credential stuffing with HTTP 429.
- **MySQL Pool Resiliency & Burst Fallback:** Configured with MySQLConnectionPool(pool_size=10, pool_reset_session=True). If traffic bursts exhaust pooled sockets, open_db() captures the exception and immediately spawns a direct on-demand connection, preventing 500 server crashes.
- **Dual-Authorization Schedule Reset:** Clearing master schedules requires dual-admin validation: one administrator submits a formal justification, and a second administrator reviews and approves the request in schedule_reset_requests.
- **Non-Repudiation Audit Trail:** All critical modifications (timetable generation, manual edits, user role changes, student approvals) are logged in audit_logs with actor IDs, roles, IP addresses, and timestamps.

6. Stakeholder Roles & Capstone Live Demonstration Script

Demonstrate the system to the defense panel in this logical 5-stage workflow:

Role	Route & Scope	Live Demonstration Actions & Key Defense Talking Points
1. Super Admin	/super_admin Global Access	Show the User Account Directory; approve pending student registrations; highlight the **Audit Logs Table** and the Dual-Authorization Schedule Reset approval queue.
2. System Admin	/dashboard /schedule	Import rooms, subjects, and instructors via CSV. Click **'Generate Schedule'** (point out sub-2s CSP execution). Open the **Interactive Conflict Resolver** to verify 0 overlaps. Demonstrate the **Publish Schedule** toggle.
3. Faculty / Proctor	/proctor/dashboard Faculty Scoped	Submit availability windows. Show assigned proctoring shifts and room assignments. **Highlight anti-self-proctoring:** show that the professor is never assigned to subjects they teach.
4. Dept Secretary	/secretary/dashboard Institute Scoped	Demonstrate **Chain-of-Custody Tracking**: transition test papers from *Pending* → *Collected* → *Returned*. Simulate a proctor absence and execute a **1-Click Standby Reliever Dispatch**.
5. Student & Guest	/student/timetable /public_timetable	Log in as a student to view personalized schedules and download the **Official Printable Examination Slip (PDF)**. Open Incognito to /public_timetable to show cross-section lookups for irregular examinees.

7. Panelist Defense Q&A; Master Guide (Anticipated Questions & Winning Answers)

Deliver these concise, authoritative technical responses during the panel questioning period:

Q1: Why did you develop a custom heuristic CSP engine instead of integrating an off-the-shelf solver like Google OR-Tools?

Answer: While Google OR-Tools is a capable solver, it introduces heavy C++ binary dependencies, high RAM consumption, and deployment fragility on lean cloud instances. Academic scheduling at CCA includes unique institutional constraints—such as sister-section synchronization, 1-hour cognitive rest intervals, anti-self-proctoring token matching, PWD room pairings, and inter-building transit buffers. Our custom CSP heuristic with a Tiered Relaxation Ladder was engineered specifically for these constraints, executing in under 2 seconds with 100% deterministic constraint satisfaction.

Q2: How does the system mathematically prevent test question leaks between different sections?

Answer: Through Sister-Section Synchronization in system/scheduler.py. The engine groups all sections taking the same course or sharing a content_group_id into an atomic scheduling cluster. It assigns all sections to the exact same date and timeslot across parallel classrooms. Because exams take place concurrently, early test-takers cannot leak questions to other sections.

Q3: How does the platform handle emergency faculty absences on examination day?

Answer: The Department Secretary dashboard features an emergency dispatch system. If a proctor reports an illness or fails to arrive, the secretary logs the absence in proctor_absence and dispatches a replacement from the relievers pool with a single click. The timetable immediately updates, and the replacement event is permanently logged in audit_logs without disturbing other testing rooms.

Q4: What happens if 500 students access the portal simultaneously when timetables are released?

Answer: The system is architected for high-concurrency bursts: (1) Nginx acts as an edge reverse proxy handling SSL termination and static asset caching; (2) Gunicorn utilizes 3 pre-forked worker processes to handle dynamic requests; (3) database connections are managed via a persistent 10-socket MySQL pool with automatic on-demand direct-connection fallback to prevent pool exhaustion; and (4) student schedule queries are read-optimized indexed SQL lookups.

Q5: How do you prevent accidental schedule deletion or malicious tampering by rogue administrators?

Answer: We engineered a Dual-Authorization Schedule Reset workflow. A System Administrator cannot clear active timetables on impulse. They must submit a formal request with a documented reason into schedule_reset_requests. The reset remains pending until a second administrator reviews and approves it, at which point an automated archive snapshot is stored in exam_archives.

Q6: Why are there separate Super Admin and System Admin roles?

Answer: This adheres to the Principle of Least Privilege and Separation of Duties. System Administrators manage day-to-day academic scheduling operations (subjects, classrooms, proctor assignments, timetable publishing). Super Administrators govern global security parameters, user account provisioning, role promotions, SMTP credentials, and dual-authorization approvals. This ensures no single operational admin can unilaterally compromise the system.

Q7: What are the current limitations of your system and what are your recommendations for future research?

Answer: Currently, proctor substitutions require a secretary to manually select from the available reliever pool; future enhancements could incorporate automated push notifications and geolocation dispatch. Additionally, while the system generates official PDF passes, integrating direct SMS gateway alerts would enhance student notification during severe weather disruptions or emergency campus closures.

8. Defense Day Operational Checklist & Contingency Strategy

- **Cloud VPS Verification:** Verify DigitalOcean Droplet status via SSH (systemctl status cca_exam.service nginx).
- **Offline Local Fallback:** Keep local XAMPP Apache/MySQL and Flask running on your laptop (run.bat) in case venue Wi-Fi drops.
- **Role Credentials Card:** Have test logins ready for Super Admin, System Admin, Department Secretary, Proctor, and Student.
- **Pre-Loaded Browser Windows:** Open distinct browser windows or profiles for each role beforehand to avoid demo login delays.
- **Panel Grace Strategy:** If panelists suggest a new constraint, answer: *'That is an excellent institutional consideration. In our architecture, that rule can be seamlessly added as an evaluation filter in our CSP heuristic ladder.'*